FAQ-03: OneAgent vs OpenTelemetry — A Decision Framework

Series: FAQ — Frequently Asked Questions | **Reference:** 03 — OneAgent vs OpenTelemetry — A Decision Framework | **Created:** May 2026 | **Last Updated:** 09/02/2026

Overview

The recurring question:

Three related questions show up repeatedly. Teams already running OpenTelemetry ask: *Is it worth the effort to convert our existing OTel-instrumented services to Dynatrace OneAgent?* And the sharper version: *If our OTel implementation is working today, do we need OneAgent at all?* And from teams starting fresh with nothing instrumented yet: *We have a clean slate — which should we install first, OneAgent or OpenTelemetry?*

All three come down to a level-of-effort versus capability-gain trade-off, and all three deserve a frank answer. The answer is the same across every runtime Dynatrace supports — Java, Kotlin, Scala, Groovy, .NET, Node.js, Python, Go, PHP, Ruby — though a handful of runtimes have edge cases (covered in §7 and §14).

Short answer (effort-vs-conversion): If an application is already emitting OpenTelemetry data and is not running on serverless, **the most cost-effective path is almost always to keep OTel for application instrumentation and add OneAgent alongside it for host, runtime, and infrastructure coverage** — not to convert. The two are complementary, not competing. A wholesale conversion is usually a *deletion* exercise, not a re-instrumentation exercise — and even then, the gain is rarely worth the disruption unless a specific Dynatrace capability is needed that OTel cannot provide on its own.

Short answer (is OneAgent required at all): No, not strictly. OTel alone can fully instrument an application and ship to Dynatrace via OTLP — Davis AI works on that data, traces correlate via W3C `traceparent`, and metrics/logs land in Grail. **But OTel-only leaves real gaps** in host metrics, runtime internals (heap / GC / threads / event loop), Smartscape topology, code-level method tracing, and RUM-to-backend stitching. Whether those gaps matter depends on the workload — see §2 for the side-by-side capability table.

Short answer (greenfield — which one first?): For a new deployment on a standard OneAgent-supported runtime (Java / .NET / Node.js / Python / PHP) running on long-lived hosts or containers, **install OneAgent first** — time-to-first-trace is measured in minutes, with zero code change, and Smartscape + Davis + framework auto-instrumentation work at full fidelity from day one. Layer OTel in later (or in parallel) when you need custom business spans or vendor-neutral telemetry; the two are designed to coexist (see §8). The exceptions where OTel comes first: **serverless** workloads (OneAgent can't run there), **Go or Ruby** application code (no OneAgent SDK exists), **async-fragmenting runtimes** like Scala effect systems (otel4s / zio-telemetry are technically required — see §7), and any **vendor-portability or multi-backend mandate** (OTel as source of truth, OneAgent as enrichment).

This FAQ frames the trade-offs, the hidden costs, and the workload-specific edge cases — including **async-fragmenting runtimes** (effect systems, coroutines, async/await without explicit context propagation) where the choice has a clear technical answer rather than a preference.

Table of Contents

Prerequisites

Requirement	Details
Audience	Engineering leads, SRE/observability architects, platform owners evaluating instrumentation strategy
Format	Decision-support FAQ — no DQL or hands-on labs
Assumed knowledge	Familiarity with the basics of distributed tracing, the difference between an *agent* and an *SDK*, and what your services run on (long-lived host/container vs. serverless)
Scope	All major runtimes Dynatrace supports — Java, Kotlin, Scala, Groovy, .NET (C# / F# / VB), Node.js (JS / TS), Python, Go, PHP, Ruby. Runtime-specific edge cases (effect systems, coroutines, async/await context propagation) are called out in §7

1. Framing the Question Correctly

Customers usually ask this as **OneAgent OR OpenTelemetry** — a binary choice. That framing is the first mistake. The two tools answer different questions:

- **OneAgent** answers: *"What is happening on my hosts, in my application runtime (JVM / CLR / V8 / CPython / Go runtime / Zend / Ruby VM), and in the frameworks I'm running — without me writing any instrumentation code?"*
- **OpenTelemetry** answers: *"How do I emit portable, vendor-neutral, code-level telemetry from my application — including the parts no agent can see automatically?"*

Re-framed correctly, the question becomes: **"Given we already have OTel, do we still need OneAgent? And if we add OneAgent, do we need to remove OTel?"** The answer to the first is *usually yes* (for non-serverless workloads). The answer to the second is *almost never*.

> **Sources:** Use OneAgent with OpenTelemetry (DT docs), Extend Dynatrace with OpenTelemetry (DT docs).

2. Is OneAgent Required at All?

Short answer: No, not strictly — but most non-serverless teams are still better off with both.

OpenTelemetry alone can fully instrument an application and ship to Dynatrace via OTLP. Traces correlate via W3C `traceparent`, metrics and logs land in Grail, and Davis AI operates on that data. For serverless workloads, async-fragmenting runtimes that lack agent-side context propagation, or a vendor-portability mandate, **OTel-only is the right answer** — OneAgent adds nothing critical, or in the case of serverless cannot run at all.

Where OTel-only leaves real gaps (and why most non-serverless teams add OneAgent anyway):

Capability	OTel-only	Requires OneAgent
Application traces / metrics / logs	Yes	—
Runtime internals (heap, GC, threads, event loop, allocations)	Partial — needs runtime-telemetry receiver wired per service (JMX for JVM, EventCounters for .NET, perf_hooks/prom-client for Node, psutil for Python, runtime/metrics for Go)	Yes — automatic

Capability	OTel-only	Requires OneAgent
Host metrics (CPU, memory, disk, network)	Partial — needs OTel Collector with hostmetrics receiver deployed on every node	Yes — automatic
Process / container correlation	Manual attribute hygiene	Yes — automatic
Smartscape topology and dependency map	Not available	Yes
PurePath code-level method tracing on demand	Not available	Yes
Real-User Monitoring ↔ backend trace stitching	Manual wiring	Yes — automatic
Auto-instrumentation of frameworks OTel doesn't cover (proprietary middleware, exotic app servers)	Gap	Yes — often covered
Davis AI fidelity	Good if attributes are clean and consistent	Best — Davis is purpose-built for OneAgent's data model
Update cadence	Per-application redeploy	Centrally managed; no app redeploy

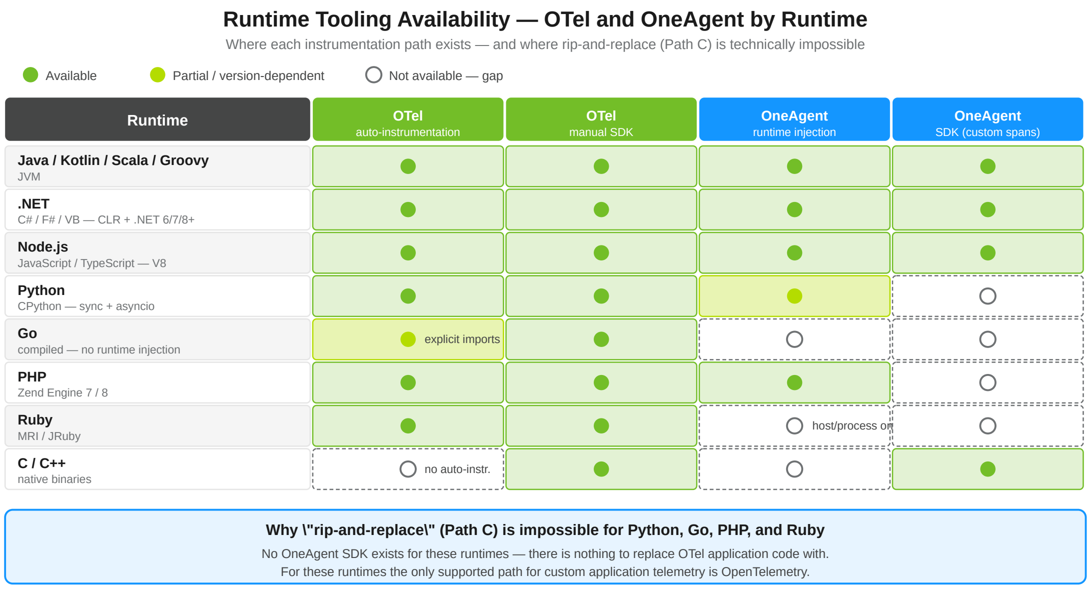
Decision rule in three lines:

- **Serverless, multi-backend mandate, or async-fragmenting runtime without OTel hooks** → OTel only. OneAgent doesn't help, or can't run.
- **Standard workloads on long-lived hosts/containers, and Smartscape + Davis at full fidelity matters** → add OneAgent alongside OTel. The install is a deployment task — no code change — and the two coexist by design (see §8 for the Span Sensor vs OTLP patterns).
- **Cost-constrained or instrumentation-fatigued, and OTel is working today** → stay OTel-only. No telemetry is missing — only some platform-level conveniences. Re-evaluate if and when those gaps cause real pain.

Honest framing: OneAgent is not *required*, it is an *accelerator*. It buys infrastructure coverage and topology that would otherwise have to be built manually with the OTel Collector, hostmetrics receivers, runtime-metrics receivers, and a clean attribute strategy. If those are already in place and working, the marginal value of adding OneAgent is much smaller — though never zero, because Smartscape, PurePath, and the Davis-AI-on-OneAgent-data path have no direct OTel equivalents.

> **Sources:**
> - Use OneAgent with OpenTelemetry (DT docs)
> - Ingest OpenTelemetry — getting started (DT docs)
> - Technology support (DT docs)
> - OneAgent SDK for Java (Dynatrace GitHub) — *"not supported on serverless code modules .. Consider using OpenTelemetry instead"*

3. What Each Tool Actually Is



Read across each row: which tooling is available for that runtime — and where the gaps make a conversion infeasible.

OneAgent is a Dynatrace-installed binary that runs on the host (or as a container sidecar/initcontainer in Kubernetes via the Dynatrace Operator). When the application runtime starts, OneAgent injects itself and auto-instruments a long list of frameworks and runtimes. It produces *PurePath* traces, host metrics, runtime metrics, log streams, process snapshots, and Smartscape topology — all without code changes. OneAgent's runtime injection covers Java/Kotlin/Scala/Groovy (JVM), .NET (CLR), Node.js (V8), PHP (Zend), and Python; for runtimes it does not inject (Go, Ruby), OneAgent still captures host/process telemetry around them.

OpenTelemetry (OTel) is a CNCF specification with a per-runtime delivery model. The two main consumption modes vary slightly per runtime:

Runtime	OTel auto-instrumentation entry point	OTel manual SDK entry point
Java / Kotlin / Scala / Groovy (JVM)	`-javaagent:opentelemetry-javaagent.jar`	`io.opentelemetry.api` (otel4s, zio-telemetry, kotlinx-coroutines wrappers)
.NET	`OpenTelemetry.Instrumentation.*` NuGet packages auto-wired in `Program.cs`, or the .NET auto-instrumentation injector	`System.Diagnostics.ActivitySource` / OTel SDK
Node.js (JS / TS)	`--require @opentelemetry/auto-instrumentations-node/register`	`@opentelemetry/api`
Python	`opentelemetry-instrument <command>` (CLI wrapper) or programmatic auto-instrumentation	`opentelemetry-api` / `opentelemetry-sdk`
Go	No bytecode injection — explicit `otelhttp`, `otelgin`, `otelgrpc`, etc. middleware imports	`go.opentelemetry.io/otel`
PHP	`opentelemetry` PECL extension + auto-instrumentation Composer packages	`open-telemetry/api` Composer package
Ruby	`opentelemetry-instrumentation-all` gem	`opentelemetry-api` / `opentelemetry-sdk`

These OTel modes are independent — you can use any combination, or none. "Already instrumented with OTel" usually means a mix: the auto-instrumentation for HTTP/DB plus some manual SDK calls for business logic.

JVM has a third delivery model — and it is the one most production teams should prefer. The auto-instrumentation column above shows the static `-javaagent:` flag, but for JVM workloads there are actually three distinct OTel delivery options, with materially different operational risk profiles:

Mode	How it loads	Operational characteristics
1. **SDK in code**	OTel dependencies in `pom.xml` / `build.gradle`; explicit `Tracer` calls in source	Tied to the application build; redeploy needed for instrumentation changes; full control over what is captured
2. **Static `-javaagent`**	JVM startup flag points at `opentelemetry-javaagent.jar` — auto-instrumentation kicks in before `main()`	Coupled to the JVM start command; if the agent JAR fails to load (bad version, classpath conflict, missing file), the JVM may fail to start, taking the line-of-business application down with it
3. **Dynamic attach**	A separate process attaches the OTel agent to a running JVM via the JVM `com.sun.tools.attach.VirtualMachine` API after the application has started	Decoupled from application startup — if the instrumentation phase fails, the JVM keeps running and serving traffic. Updates and rollbacks are reversible without touching the application

Recommended for production: Option 3 (dynamic attach). Option 2 places the OTel agent on the critical path of JVM startup — a failed agent load can prevent the JVM from starting and take the workload offline. Option 3 attaches *after* the JVM is live, so a failed instrumentation phase leaves the application running unaffected. The trade-off is operational complexity: you need a sidecar, init container, or platform mechanism that performs the attach.

This is the same model **Dynatrace OneAgent** uses by design — OneAgent's Java code module injects via JVMTI and platform-level hooks against an already-running JVM, never via boot-time `-javaagent`. It is also why OneAgent updates do not require application redeploys: a new agent version attaches at the next workload restart (or via runtime hot-attach in some configurations) without touching the application's start command.

Dynatrace OneAgent SDKs (separate, narrow libraries — *not* OneAgent itself) exist per language for adding custom traces inside an application that is *already running under OneAgent*. With no OneAgent attached, the SDK calls become no-ops.

Runtime	OneAgent SDK	Status
Java	OneAgent-SDK-for-Java	Active — version 1.9.0; explicitly recommends OpenTelemetry for serverless
.NET	OneAgent-SDK-for-DotNet	GA — .NET Standard 1.0 (Full Framework 4.5+ / .NET Core 1.0+); v1.8.0 last release
Node.js	OneAgent-SDK-for-NodeJs	Active
C / C++	OneAgent-SDK-for-C	Active
Python	None (OneAgent injects automatically; OTel SDK recommended for custom spans)	—
Go	None (OneAgent observes Go via host/process; OTel SDK recommended for custom spans)	—
PHP	None (OneAgent's PHP code module covers auto-instrumentation; OTel SDK recommended for custom spans)	—
Ruby	None (Dynatrace recommends the Ruby OpenTelemetry stack for code-level traces)	—

Implication: for Python, Go, PHP, and Ruby, "convert custom OTel spans to OneAgent SDK" is not even an option — the SDK doesn't exist. The conversation in those runtimes is automatically "layer OneAgent (where it can run) alongside the existing OTel spans," not "rewrite the OTel spans."

> **Sources:**
> - OneAgent SDK for Java (Dynatrace GitHub) — v1.9.0; serverless-not-supported guidance
> - OneAgent SDK for .NET (Dynatrace GitHub) — *"the SDK is built using .NET Standard 1.0"*; 1.8.0 is the newest version in both the release-notes and support-status tables, and the metrics surface was withdrawn across two releases — *"1.7.1 Deprecates metrics-related types and APIs"*, then *"1.8.0 Removes deprecated APIs and types."*
> - OneAgent SDK for Node.js (Dynatrace GitHub)
> - OneAgent SDK for C/C++ (Dynatrace GitHub)
> - Java technology support (DT docs)

> - VirtualMachine (Oracle JDK 21) — `loadAgent()` API for dynamic JVM attach
> - opentelemetry-java-instrumentation (OTel GitHub)
> - @opentelemetry/auto-instrumentations-node (OTel JS contrib)
> - opentelemetry-python-contrib (OTel GitHub)
> - opentelemetry-go-contrib (OTel GitHub)
> - opentelemetry-php (OTel GitHub)
> - opentelemetry-ruby (OTel GitHub)

4. Capability Comparison

Dimension	OneAgent	OpenTelemetry (auto-instrumentation + SDK)
Owner	Dynatrace (vendor)	CNCF / OpenTelemetry community (vendor-neutral)
Backend	Locked to Dynatrace	Any OTLP-compatible backend (Dynatrace, Jaeger, Tempo, Datadog, New Relic, Splunk, Honeycomb, Grafana Cloud, etc.)
Runtime model	Native binary on host or in container; injects into supported runtimes at startup	Per-runtime: javaagent, .NET injector, Node `--require`, Python `opentelemetry-instrument`, explicit Go imports, PECL extension, Ruby gem
Signals captured	Traces, runtime metrics, host metrics, process metrics, log streams, topology, real-user, deep code-level (PurePath)	Traces, metrics, logs, baggage, context (signals you choose to emit)
Auto-instrumented frameworks	Hundreds across JVM, .NET, Node.js, PHP, Python — Spring / ASP.NET / Express / Laravel / Django, plus all major SQL drivers, message brokers, web servers, app containers	Per-runtime list — Spring / ASP.NET / Express / Django / Gin / Rails / Laravel etc.; vary in completeness across runtimes (JVM and .NET have the broadest OTel coverage; Go requires explicit imports; Ruby and PHP are still maturing)
Host / runtime / process metrics	Yes — included automatically	Only if you add the host-metrics or runtime-metrics receiver via an OTel Collector / node-exporter; not part of basic SDK
Smartscape / topology / dependency map	Yes — automatic	Not provided by OTel; can be approximated from spans but no real-time topology graph
Davis AI / problem detection / RCA	Yes — built into Dynatrace platform on OneAgent telemetry	Available when OTel data is ingested into Dynatrace; quality depends on attribute completeness
Serverless — AWS Lambda, GCP Cloud Functions	Not supported (OneAgent SDK README + per-runtime guidance)	Fully supported across all runtimes
Serverless — Azure Functions	**Plan- and runtime-dependent — not a blanket no.** OneAgent 1.343 (released 07/28/2026) adds Azure Functions support for **Python on the Flex Consumption Plan (Linux)** and for **Java and Node.js on Windows plans**, across multiple trigger types. Outside those plan/runtime combinations, still unsupported. Verify the OneAgent version actually deployed on the Function.	Fully supported across all runtimes
Update model	Centrally managed by Dynatrace; no app redeploy	Application redeploy required for SDK; auto-instrumentation update requires process restart
Runtime version floor	Per-runtime; e.g. JVM 8+, .NET Framework 4.5+ / .NET Core 2.1+ / .NET 5+, Node 20+, PHP 7.1+, Python 3.8+ — see Dynatrace supported-technologies matrix for current detail	Per-runtime; e.g. Java 8+, .NET 6+ (most current), Node 14+, Python 3.8+, Go 1.21+ (current OTel Go), PHP 8.0+, Ruby 3.0+
Code change required	None for auto-coverage	None for auto-instrumentation; explicit imports for custom spans/metrics

Dimension	OneAgent	OpenTelemetry (auto-instrumentation + SDK)
Async-context fragmentation risk	Thread-local-style propagation — fragments under fibers (effect-system Scala), some coroutine setups, and async/await without explicit context plumbing	Runtime-aware libraries (otel4s, zio-telemetry, kotlinx-coroutines instrumentation, asyncio context, AsyncLocalStorage, AsyncLocal, context.Context) handle each runtime's async model correctly
Release cadence	Monthly (Dynatrace SaaS sprints)	Per-runtime: monthly minor releases for Java / .NET / Node / Python; Go versions independently

The serverless rule is no longer one rule

"OneAgent does not run on serverless" was accurate as a single statement until OneAgent 1.343. It now has to be split by platform — and this matters more here than it would in a reference table, because §9's decision tree treats serverless as a first-question gate, and a wrong answer at a gate misroutes a reader for the life of the project.

- **AWS Lambda and GCP Cloud Functions** — unchanged. OTel is the answer, and the per-runtime OneAgent SDK READMEs direct serverless users there explicitly.
- **Azure Functions** — now depends on the **hosting plan** and the **runtime**, not on "is it serverless". OneAgent 1.343 covers Python on the Flex Consumption Plan (Linux), and Java and Node.js on Windows plans. Any other plan tier or runtime remains OTel-only.
- **And still OTel until the agent is actually there.** On the covered Azure combinations, OTel remains the working answer until the OneAgent version deployed on the Function reaches 1.343. **Tenant version is not agent version** — verify the deployed agent rather than inferring support from your tenant's sprint.

The same release also adds OneAgent auto-tracing for **Azure Service Bus, Event Hub, and Cosmos DB** (Java, Node.js, Python). That bears on this decision for a second-order reason: a Function is often thin, and the spans worth having are the messaging and database calls it makes. Where those dependencies were the reason a team reached for OTel, 1.343 changes the calculus even on Function runtimes it does not cover.

> **Sources:**
> - OneAgent 1.343 release notes (DT docs) — released 07/28/2026; Azure Functions support for Python on the Flex Consumption Plan (Linux) and Java / Node.js on Windows plans, multiple trigger types; auto-tracing for Azure Service Bus, Event Hub, and Cosmos DB (Java, Node.js, Python)
> - OpenTelemetry Java SDK (OpenTelemetry GitHub) — monthly minor releases; Java 8+ floor; current stable 1.62.0
> - OpenTelemetry main site (opentelemetry.io)
> - Use OneAgent with OpenTelemetry (DT docs)
> - OneAgent SDK for Java (Dynatrace GitHub) — serverless-not-supported; the basis for the AWS Lambda / GCP Cloud Functions row
> - **Derived:** the "verify the deployed agent, not the tenant sprint" qualifier follows from agent fleets upgrading independently of tenant version

5. What "Convert to OneAgent" Really Means

When customers say "convert from OTel to OneAgent", they often imagine a 1:1 re-instrumentation effort comparable to the original OTel rollout. **It is not.**

For workloads where OneAgent's automatic coverage is sufficient (the common case — REST/gRPC services on Spring, ASP.NET Core, Express, Django, Rails, plain thread pools or default async runtime), "converting" looks like this:

1. **Install OneAgent** on the host or via the Dynatrace Operator on Kubernetes. Restart the processes. Done — instrumentation is live.
2. **Optionally remove the OTel auto-instrumentation** (the `-javaagent:`, the .NET injector, the Node `--require`, the Python `opentelemetry-instrument`, the explicit Go middleware imports). This step is *deletion*, not authoring.
3. **Decide what to do with custom OTel spans/metrics/logs** in code:
   - **Easiest path:** leave them in place — Dynatrace ingests OTel data via OTLP, and OneAgent's *OpenTelemetry Span Sensor* (where supported) stitches them into the same trace as OneAgent's auto-instrumented spans. (Caveat: do not enable both Span Sensor and OTLP export simultaneously, or you will get duplicate spans.)
   - **Replacement path:** rewrite each `Tracer` / `Meter` / `Logger` call against the OneAgent SDK for your runtime — *if one exists* (Java, .NET, Node.js, C/C++ only). For Python, Go, PHP, and Ruby, no OneAgent SDK exists, so this path is not available; OTel is the supported way to emit custom application telemetry. And even where the OneAgent SDK exists, it is intentionally narrow (incoming/outgoing remote calls, custom services, web requests, messaging, SQL, custom request attributes) and **does not cover**

metrics or logs at all — replacing OTel with the OneAgent SDK necessarily *loses* the metrics/logs surface unless you separately ingest them via Dynatrace metrics ingest APIs.

For workloads where OneAgent's automatic coverage is insufficient (async-fragmenting runtimes, exotic frameworks, message brokers OneAgent doesn't auto-instrument), removing OTel is actively harmful — you would be deleting working instrumentation and replacing it with nothing.

The realization: "Convert to OneAgent" is rarely a re-instrumentation project. It is an *agent installation* project, possibly followed by *deleting OTel auto-instrumentation* if you want to simplify the runtime stack. The custom OTel code in the application generally stays exactly where it is.

> **Sources:** Use OneAgent with OpenTelemetry (DT docs) — *"create duplicate spans"* warning; OpenTelemetry Span Sensor mechanism, OneAgent SDK for Java (Dynatrace GitHub), OneAgent SDK for .NET (Dynatrace GitHub) — the release notes record *"1.7.1 Deprecates metrics-related types and APIs"* and then *"1.8.0 Removes deprecated APIs and types."* — leaving the SDK with no metrics or logs surface.

6. Framework Auto-Instrumentation Coverage Across Runtimes

Coverage is the single biggest factor in whether *adding* OneAgent gets you what you want or leaves you patching gaps with the SDK or OTel. The table below summarizes the overlap by runtime — full per-language support matrices link from the `> **Sources:**` block at the end of this section.

Runtime	Web framework / RPC coverage	Database / cache / queue coverage	Notes on gaps
Java / JVM (Java, Kotlin, Scala, Groovy, Clojure)	OneAgent: Spring (MVC + Boot + WebFlux), Servlet / JAX-RS, Akka HTTP 10.1+, Play 2.6+, Tomcat / Jetty / WebLogic / WebSphere / JBoss, gRPC, OkHttp, Apache HttpClient, Java 11+ HttpClient. OTel: similar list plus Akka Actors 2.3+, Finatra 2.9+, Scala ForkJoinPool 2.8+	Both: all major JDBC drivers, Kafka, RabbitMQ / AMQP, JMS, ActiveMQ, Redis, Cassandra, MongoDB, Elasticsearch	Neither auto-instruments **http4s**, **Tapir**, **fs2 streams**, **Monix**, or **Cats Effect IO** — these are async-fragmenting and need runtime-aware OTel libs (see §7)
.NET (C#, F#, VB.NET — CLR + .NET 6 / 7 / 8+)	OneAgent: ASP.NET Framework, ASP.NET Core (Kestrel + IIS), WCF, gRPC, HttpClient, named-pipe / TCP / MSMQ services. OTel: ASP.NET Core, HttpClient, gRPC, SignalR via `OpenTelemetry.Instrumentation.*` packages	Both: ADO.NET (SQL Server, Oracle, MySQL, Postgres, SQLite), Entity Framework Core, RabbitMQ, Azure Service Bus, Redis (StackExchange.Redis; **ServiceStack.Redis** added in OneAgent 1.343 — verify the agent version deployed), MongoDB	OneAgent auto-instruments classical ASP.NET Framework; OTel coverage of ASP.NET Framework is narrower (mostly ASP.NET Core onwards)
Node.js (JavaScript / TypeScript — V8)	OneAgent: Express, Koa, Hapi, Fastify (newer), NestJS, native HTTP/HTTPS, gRPC. OTel: same list via `@opentelemetry/auto-instrumentations-node` (Express, Koa, Hapi, Fastify, Restify, Connect, GraphQL, gRPC, Apollo)	Both: pg, mysql / mysql2, mongodb, redis, ioredis, kafkajs, amqplib, aws-sdk, elasticsearch	OneAgent and OTel both rely on patching `require()` — module versions outside the supported range are skipped silently
Python (CPython — async-aware)	OneAgent: Django, Flask, FastAPI (recent versions), Tornado, Starlette, ASGI, gunicorn, uWSGI. OTel: similar list via `opentelemetry-instrumentation-django` / `flask` / `fastapi` / `aiohttp` / `tornado` / `pyramid` etc.	Both: psycopg / psycopg2 / asyncpg, PyMySQL / mysqlclient / mysql-connector, MongoDB (pymongo), Redis (redis-py), Kafka (kafka-python / confluent-kafka), Celery, SQLAlchemy	OneAgent's Python module is more conservative on async frameworks; OTel covers asyncio-native coverage more aggressively
Go (compiled — no runtime injection)	OneAgent: observes Go binaries via host/process telemetry; **does not auto-instrument Go application code**. OTel: explicit imports — `otelhttp` (net/http), `otelgin`, `otelmux`, `otelchi`, `otelfiber`, `otelgrpc`, `otelecho`	OneAgent: none for Go libraries. OTel: `otelsql` (database/sql wrapper), `otelpgx`, `otelgorm`, `otelmongo`, `otelredis`, `otelkafka`, `otelsarama`	**Go is OTel-first by design** — no `-javaagent`-equivalent and no Go OneAgent SDK. OneAgent still adds value via host/process/Smartscape, but application spans must come from OTel

Runtime	Web framework / RPC coverage	Database / cache / queue coverage	Notes on gaps
PHP (Zend Engine 7 / 8)	OneAgent: Apache mod_php / php-fpm, Laravel, Symfony, CodeIgniter, WordPress (with the PHP code module). OTel: `open-telemetry/opentelemetry-auto-instrumentation` Composer packages — Laravel, Symfony, Slim, WordPress, CakePHP	Both: PDO, mysqli, mongodb, Redis, Memcached. OTel additionally: Guzzle HTTP client	OneAgent's PHP coverage is mature; OTel PHP auto-instrumentation is newer and library-by-library
Ruby (MRI, JRuby)	OneAgent: limited Ruby support — primarily host/process telemetry. OTel: `opentelemetry-instrumentation-rails` , `sinatra` , `rack` , `grape`	Both: ActiveRecord, mysql2, pg, redis, mongo. OTel: also `opentelemetry-instrumentation-net_http` , `faraday`	**Ruby is OTel-first** — Dynatrace's recommended path for code-level Ruby tracing is OpenTelemetry, with OneAgent supplying host/process/Smartscape

Patterns to notice:

- **Overlap zone** (most teams gain immediate value from OneAgent) — Java, .NET, Node.js, Python, PHP web stacks: standard frameworks + standard SQL drivers + standard message brokers. OneAgent + OTel duplicates effort here, which is why the §8 Span Sensor / OTLP coexistence pattern exists.
- **OneAgent-only coverage** — Smartscape topology, RUM-to-backend stitching, on-demand PurePath capture, host/runtime metrics without a Collector. These are platform constructs, not signals OTel emits.
- **OTel-only coverage** — Go application code, Ruby application code, async-fragmenting runtimes (effect systems, certain coroutine layouts — see §7), most serverless platforms.
- **Coverage gaps in both** — exotic frameworks (http4s, Tapir, niche message brokers, custom RPC layers). These need either a community OTel instrumentation library, manual instrumentation, or both.

> **Sources:**
> - Java technology support (DT docs) — OneAgent JVM framework coverage (Spring, Servlet/JAX-RS, Akka HTTP 10.1+, Play 2.6+, Tomcat/Jetty/WebLogic/WebSphere/JBoss, gRPC)
> - Technology support matrix (DT docs)
> - supported-libraries (opentelemetry-java-instrumentation) — kotlinx.coroutines 1.0+, Akka HTTP 10.0+, Akka Actors 2.3+, Finatra 2.9+, Scala ForkJoinPool 2.8+
> - @opentelemetry/auto-instrumentations-node (OTel JS contrib)
> - opentelemetry-python-contrib (OTel GitHub)
> - opentelemetry-go-contrib (OTel GitHub) — `otelhttp` / `otelgin` / `otelmux` / `otelchi` / `otelfiber` / `otelgrpc` / `otelecho` / `otelsql` / `otelpgx` / `otelgorm` / `otelmongo` / `otelredis` / `otelsa`

7. Async Context Propagation Across Runtimes

Async Context Propagation — Primitives by Runtime

Each runtime carries trace context in a different primitive — does each tool propagate it correctly?

● Propagates correctly ● Mixed / version-dependent ○ Fragments / not applicable

Runtime + async model	Context primitive	OneAgent	OTel (right library)
Java (classical) threads + executors + CompletableFuture	ThreadLocal	●	●
Java 21+ (Project Loom) virtual threads on carrier threads	ScopedValue	●	●
Kotlin coroutines cooperative coroutines on threads	CoroutineContext	● verify version	●
Scala (classical) Akka, Play, Future / ForkJoinPool	ThreadLocal	●	●
Scala (effect systems) ZIO, Cats Effect, fs2, http4s on IO	IOLocal / FiberRef	○ **fragments**	● otel4s / zio-tel
.NET (async / await) Tasks on thread pool	AsyncLocal<T>	●	●
Node.js single-threaded event loop	AsyncLocalStorage / async_hooks	●	●
Python (asyncio) cooperative coroutines on event loop	contextvars.ContextVar	●	●
Go goroutines + explicit context	context.Context (function arg)	○ no Go injection	●
PHP request-per-process	request-scoped globals	●	●
Ruby threads + fibers (Async gem)	fiber-local state	○ no Ruby app injection	●

Fragmenting cells (white, dashed) are where tracing breaks unless you switch to a runtime-aware OTel library.
Scala effect systems are the canonical case: OneAgent fragments, otel4s / zio-telemetry propagate via IOLocal / FiberRef.

This is where the "OneAgent or OTel" choice has a clear-cut technical answer in some runtimes — not a preference.

The mechanic: classical agent-based tracing (both OneAgent's auto-instrumentation and OTel's auto-instrumentation in most runtimes) propagates trace context using a runtime thread-local construct. As long as a request stays on a single thread (or the agent has hooked the thread-pool / scheduling primitives the application uses), context flows correctly. Some runtimes break this assumption.

Per-runtime async-context primitives — and how each tool handles them:

Runtime	Async / concurrency model	Context primitive	OneAgent	OTel (with right library)
Java (classical)	OS threads + thread pools + `CompletableFuture`	`ThreadLocal`	Yes — auto-instruments common executors	Yes — Java agent hooks executors
Java 21+ (Project Loom virtual threads)	Many lightweight virtual threads on few carrier threads	`ScopedValue` / virtual-thread-local	Yes — current OneAgent supports virtual threads	Yes — current OTel Java agent supports virtual threads
Kotlin coroutines	Cooperative coroutines on threads	`CoroutineContext`	Mixed — historically a gap; newer code modules cover the standard `Dispatchers`	Yes — `kotlinx.coroutines` OTel instrumentation
Scala — classical (Akka, Play, plain `Future` / `ForkJoinPool`)	OS threads / executor service	`ThreadLocal`	Yes — strong auto-instrumentation	Yes — OTel Java agent
Scala — effect systems (ZIO, Cats Effect, fs2, http4s on IO)	Lightweight fibers multiplexed onto a small thread pool; runtime explicitly does *not* preserve `ThreadLocal` across fiber suspension	`IOLocal` (Cats Effect) / `FiberRef` (ZIO)	**No** — fragments across fiber boundaries	**Yes** — *otel4s* uses `IOLocal`; *zio-telemetry* uses `FiberRef`; both propagate correctly

Runtime	Async / concurrency model	Context primitive	OneAgent	OTel (with right library)
.NET (`async` / `await`)	Tasks scheduled on a thread pool with logical-call-context flow	`AsyncLocal<T>`	Yes — OneAgent flows context through `AsyncLocal`	Yes — `System.Diagnostics.Activity` uses `AsyncLocal` natively
Node.js	Single-threaded event loop with callback / Promise chains	`AsyncLocalStorage` (built on `async_hooks`)	Yes — OneAgent's Node module hooks `async_hooks`	Yes — `@opentelemetry/context-async-hooks`
Python (asyncio)	Cooperative coroutines on an event loop	`contextvars.ContextVar`	Yes (recent Python code module)	Yes — `opentelemetry-api` uses `contextvars`
Python (greenlet / gevent / eventlet)	Cooperative greenlets monkey-patching the stdlib	Library-specific	Mixed	Mixed — OTel has per-library shims
Go	Goroutines with explicit `context.Context` plumbing	`context.Context` (passed as a function argument by convention)	N/A (OneAgent does not inject Go)	Yes — every OTel Go library takes a `context.Context`
PHP	Synchronous request-per-process model (mostly)	Request-scoped globals	Yes — OneAgent's PHP module covers it	Yes — OTel PHP covers it
Ruby	OS threads + (in newer Rubies) fibers; `Async` gem and ractors are evolving	Fiber-local state	N/A in code (Dynatrace recommends OTel for Ruby app code)	Yes — `OpenTelemetry::Context`

Symptoms when context propagation breaks (in any runtime):

- A trace begins on a request, then "loses" context after the first asynchronous boundary
- Spans appear as orphans (no parent) downstream of the boundary
- Database calls, HTTP calls, and message publishes inside an async block do not link back to the parent request span
- The trace waterfall looks plausibly populated but is structurally wrong

Practical guidance by runtime:

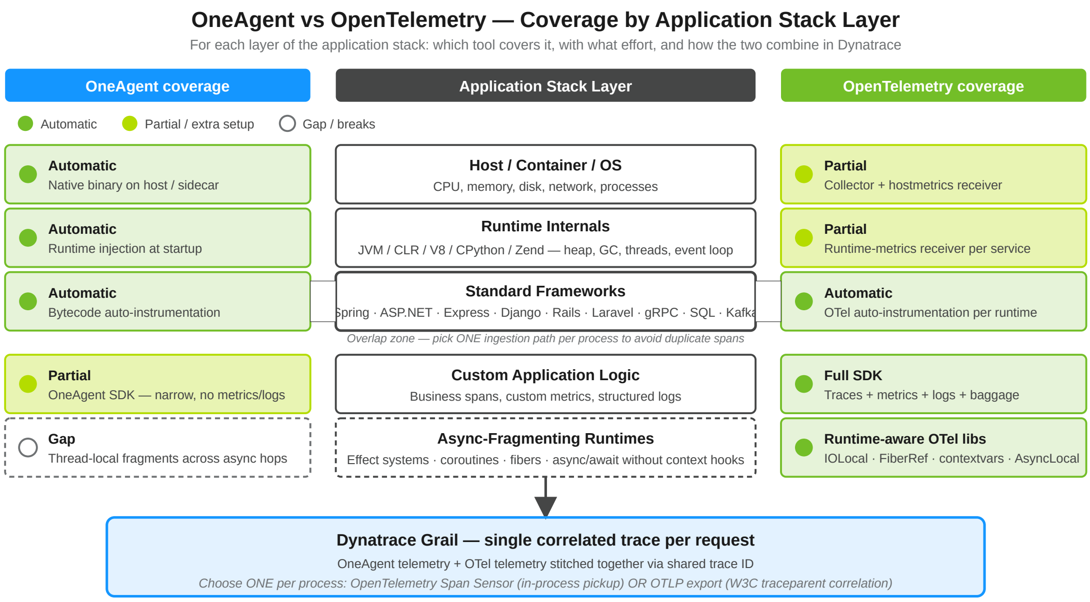
Runtime + scenario	App-level tracing	Infra-level coverage
Java / .NET / Node / Python — standard frameworks on default scheduler	OneAgent auto-instrumentation (no code change)	OneAgent (same)
Java with virtual threads	OneAgent or OTel Java agent (current versions)	OneAgent
Kotlin coroutines	OTel `kotlinx.coroutines` instrumentation if OneAgent's coroutine support is incomplete in your environment	OneAgent
Scala — Akka, Play, plain `Future` / `ForkJoinPool`	OneAgent auto-instrumentation	OneAgent
Scala — ZIO + zio-http / sttp on ZIO	**zio-telemetry (OTel)** — OneAgent fragments; SDK has no `FiberRef` model	OneAgent for host/runtime
Scala — Cats Effect + http4s + fs2	**otel4s (OTel)** — OneAgent fragments; SDK has no `IOLocal` model	OneAgent for host/runtime
Mixed Scala estate (some Akka, some ZIO)	Per-service: OneAgent for Akka/Play, OTel for effect systems	OneAgent everywhere
Go	**OTel only** at app level — no Go OneAgent SDK exists	OneAgent for host/process/Smartscape

Runtime + scenario	App-level tracing	Infra-level coverage
Ruby	**OTel only** at app level — Dynatrace's documented recommendation	OneAgent for host/process
Serverless any runtime	**OTel only** — OneAgent does not run on Lambda / Functions / Cloud Functions	—

The principle: if the runtime has an async model that fragments thread-local storage (effect systems, certain coroutine layouts, or any runtime where context is fiber-scoped or scope-of-task), use the OTel library that targets that runtime's context primitive. The OneAgent SDK has a thread-local-style context model in every language where it exists, and will fragment in the same way unless the agent itself has been updated to hook the runtime's async primitives.

> **Sources:**
>
> - AsyncLocal<T> (Microsoft Learn) — *"ambient data local to a given asynchronous control flow"*
> - AsyncLocalStorage (Node.js docs) — built on `node:async_hooks`
> - contextvars (Python docs) — natively supported in `asyncio` (Python 3.7+)
> - otel4s (Typelevel GitHub) — *"design goal is to fully and faithfully implement the OpenTelemetry Specification atop Cats Effect"*
> - zio-telemetry (ZIO GitHub)
> - kotlinx-coroutines instrumentation (OTel Java)
> - supported-libraries (opentelemetry-java-instrumentation)
> - Go OpenTelemetry walkthrough (DT docs)

8. The Coexistence Pattern (How They Work Together in Dynatrace)



Read-down logic: where the layer sits in your application stack determines which tool covers it best — and where the two must combine.

Dynatrace is explicit that the two are designed to coexist. From the Dynatrace OneAgent + OpenTelemetry documentation: *"You can implement this service by service, adopting the open standards of OpenTelemetry where openness matters most, while leveraging enhanced OneAgent features available where you need them."*

The mechanism — OpenTelemetry Span Sensor: when this code-module setting is enabled (where supported per runtime), OneAgent detects in-process OpenTelemetry API calls and weaves them into the same trace as OneAgent's auto-instrumented spans. Per the Dynatrace docs: *"These auto-instrumented spans are woven together with your manual OpenTelemetry spans into a single trace."* The result is a unified PurePath-style trace that contains both OneAgent and OTel-emitted spans, correlated through a shared trace ID.

Critical caveat: if you enable the OpenTelemetry Span Sensor *and* simultaneously export OTLP from the same process, **you will get duplicate spans** — once from the Span Sensor's in-process pickup, once from the OTLP exporter. Pick one ingestion path per process.

The two supported patterns are:

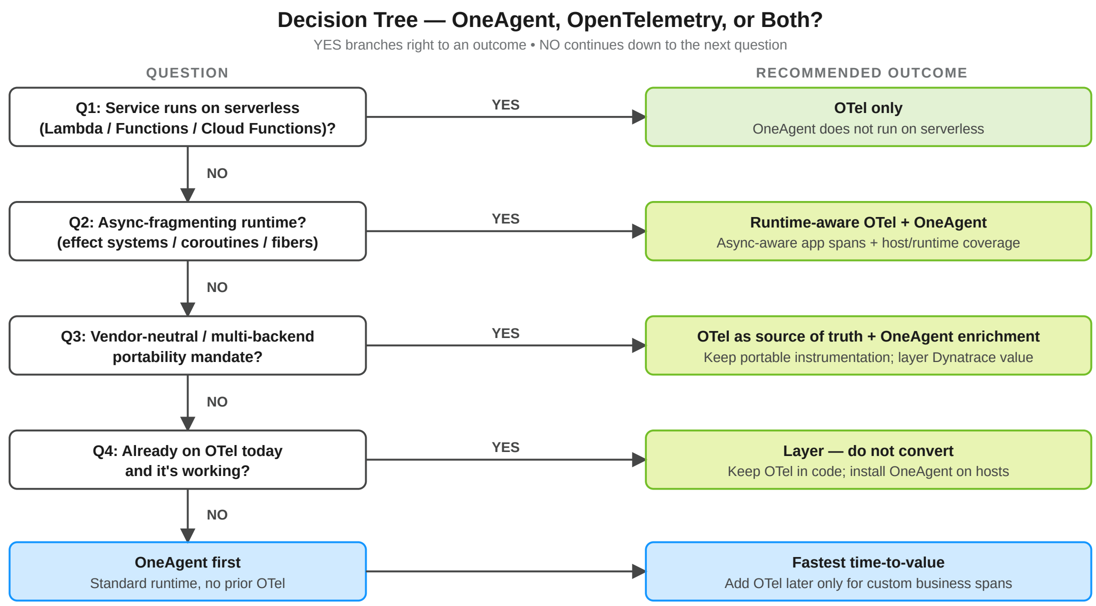
1. **Span Sensor only** — OTel API calls in code, no OTLP exporter configured. OneAgent picks them up locally, ships them inside its native channel.
2. **OTLP only** — Span Sensor disabled, OTel exporter configured to send to a Dynatrace OTLP endpoint (or to an OTel Collector forwarding to Dynatrace). OneAgent traces and OTel traces correlate via standard W3C `traceparent` propagation.

Either pattern produces a single, correlated trace in Dynatrace. Customers regularly run both OneAgent and OTel in the same process precisely because the platform was built to merge them. For runtimes where OneAgent does not inject (Go, Ruby) or for serverless workloads, Pattern 2 (OTLP) is the only option.

Span Sensor runtime support: the Dynatrace coexistence docs document Span Sensor support for **Java, Go, Node.js, PHP, and .NET** code modules.

> **Sources:**
> - Use OneAgent with OpenTelemetry (DT docs) — Span Sensor mechanism, *"woven together .. single trace"*, duplicate-spans warning, supported on Java/Go/Node.js/PHP/.NET
> - Extend Dynatrace with OpenTelemetry (DT docs) — *"adopting the open standards of OpenTelemetry where openness matters most"*
> - Ingest OpenTelemetry — getting started (DT docs) — OTLP endpoints, Collector, Istio/Envoy paths
> - W3C Trace Context (opentelemetry-specification) — `traceparent` propagation

9. Decision Tree by Workload Type



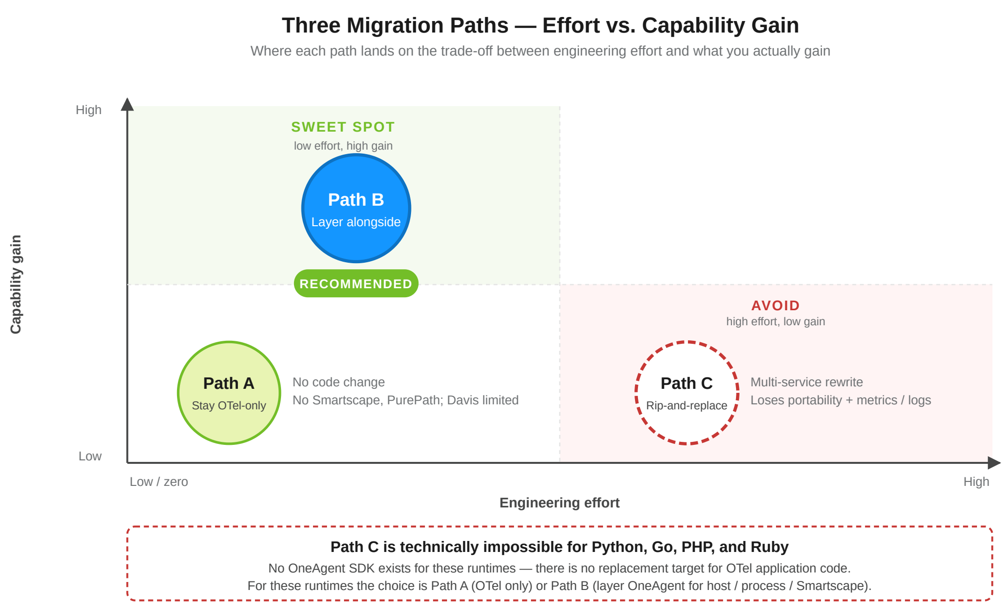
Plain-language flow:

1. **AWS Lambda or GCP Cloud Functions?** → OTel only. OneAgent does not run there, and the per-runtime OneAgent SDK READMEs explicitly direct serverless users to OpenTelemetry.
2. **Azure Functions?** → **Ask two more questions, not one.** As of **OneAgent 1.343 (released 07/28/2026)** Azure Functions is no longer a blanket no: **Python on the Flex Consumption Plan (Linux)** and **Java and Node.js on Windows plans** are supported, across multiple trigger types. So the gate is *plan* and *runtime*, not "is it serverless."
   - Outside those combinations → OTel, as before.
   - Inside them → OneAgent is an option, **once the OneAgent version deployed on the Function actually reaches 1.343.** Tenant version is not agent version; verify the deployed agent. Until it does, OTel remains the working path.
   - Either way, note that 1.343 also adds OneAgent auto-tracing for **Azure Service Bus, Event Hub, and Cosmos DB** (Java, Node.js, Python). If the reason you wanted tracing was the Function's *dependencies* rather than the Function itself, that may be the more consequential half of the release for you.

3. **Async-fragmenting runtime?** (effect systems, coroutines without agent hooks, async without explicit context propagation) → OTel via the runtime-aware library — *otel4s / zio-telemetry* (Scala effect systems), `kotlinx.coroutines` OTel instrumentation (Kotlin), `contextvars` (Python asyncio), `AsyncLocal` (.NET — handled natively), `AsyncLocalStorage` (Node.js — handled natively), `context.Context` (Go — handled natively). Add OneAgent for host/runtime signals if the workload runs on long-lived hosts or containers.
4. **Multi-backend mandate, regulatory portability, or active exit-ramp planning?** → OTel as your primary instrumentation. OneAgent becomes value-add, not the source of truth.
5. **Runtime is Go or Ruby?** → OTel for application spans (no OneAgent SDK exists for these runtimes). OneAgent still adds value through host/process telemetry and Smartscape.
6. **Standard web stack?** (Spring, ASP.NET Core, Express, Fastify, Django, FastAPI, Rails, Laravel, etc.) → OneAgent's auto-instrumentation gives you the fastest time-to-value. If you already have OTel, keep it for custom spans; if not, you may not need it at all.
7. **Already instrumented with OTel and it's working?** → Add OneAgent alongside. Do *not* rip out OTel. The combined cost is lower than the conversion cost, and you keep your portability.

> **Note on the diagram.** The SVG above still shows serverless as a single gate. Read steps 1 and 2 as the authoritative form of that branch until the graphic is redrawn.
>
> **Sources:**
>
> - OneAgent 1.343 release notes (DT docs) — released 07/28/2026; Azure Functions support for Python on the Flex Consumption Plan (Linux) and Java / Node.js on Windows plans; Azure Service Bus / Event Hub / Cosmos DB auto-tracing
> - OneAgent SDK for Java (Dynatrace GitHub) — *"not supported on serverless code modules .. Consider using OpenTelemetry instead"*
> - OneAgent SDK for Node.js (Dynatrace GitHub)
> - OpenTelemetry on AWS Lambda (DT docs) — Lambda extension + OTel; Python, Node, Java
> - Use OneAgent with OpenTelemetry (DT docs)
> - otel4s (Typelevel GitHub), zio-telemetry (ZIO GitHub)

10. Three Migration Paths and Their Effort



Position on the chart shows each path's location on the effort × gain trade-off — Path B is the only option that wins on both axes.

Path A: Stay OTel-only (no OneAgent at all)

Aspect	Reality
Effort	Zero — keep current state

Aspect	Reality
Code changes	None
What you keep	Vendor neutrality, runtime-aware tracing on async-fragmenting stacks, serverless coverage, single instrumentation model across polyglot stacks
What you give up	Smartscape topology, automatic runtime/host/process metrics without a Collector, on-demand code-level method tracing, deepest Davis AI fidelity, RUM-to-backend stitching unless you wire it manually
Best for	Greenfield CNCF-aligned organizations; serverless-heavy estates; multi-cloud/multi-vendor strategies; Go/Ruby-dominant or effect-system-dominant teams

Path B: Add OneAgent alongside OTel (the layered pattern — recommended for most)

Aspect	Reality
Effort	Low — install OneAgent on hosts or via Dynatrace Operator, restart the application processes
Code changes	None — keep all existing OTel code
What you gain	Host/runtime metrics, Smartscape, Davis AI, automatic framework coverage where it overlaps with OTel, RUM stitching
What you must decide	Span Sensor vs OTLP export (pick one per process — see §8 caveat)
Risk	Duplicate spans if both ingestion paths active simultaneously; resolved by configuration
Best for	Teams already on OTel who want Dynatrace's infra and AI features without losing their portable instrumentation

Path C: Rip-and-replace OTel with OneAgent (and OneAgent SDK for custom spans)

Aspect	Reality
Effort	Highest — install OneAgent, remove OTel auto-instrumentation + dependencies, rewrite every custom span / metric / log against the OneAgent SDK
Code changes	Per-service rewrite of every `Tracer.spanBuilder(...)` / `ActivitySource.StartActivity(...)` / equivalent, plus every metric and log emission to OneAgent SDK equivalents — and the SDK does not cover metrics or logs at all, so those signals are lost or must be re-emitted via Dynatrace metrics ingest APIs
Availability	OneAgent SDK exists only for Java, .NET, Node.js, and C/C++. **Path C is not even an option for Python, Go, PHP, or Ruby**
What you gain	One instrumentation model in code; tighter native PurePath semantics in places
What you lose	Vendor neutrality, broad community ecosystem, runtime-aware libraries on async-fragmenting stacks, serverless coverage, parts of your metrics/logs surface
Risk	High — coordinated multi-service rewrite, regression-prone, must be repeated for every new service
Best for	Almost no one. Defensible only if (a) the engineering org has a strict single-vendor mandate, (b) all workloads are non-serverless and on a runtime where the OneAgent SDK exists, (c) no async-fragmenting runtimes are involved, and (d) the cost of dual maintenance demonstrably exceeds the rewrite cost

> **Sources:** OneAgent SDK for Java (Dynatrace GitHub), OneAgent SDK for .NET (Dynatrace GitHub) — *"Removes deprecated APIs"* (v1.8.0); no metrics/logs surface, Use OneAgent with OpenTelemetry (DT docs).

11. Pros and Cons Side-by-Side

OneAgent — pros

- Zero-code instrumentation for hundreds of frameworks across Java, .NET, Node.js, Python, PHP
- Host, runtime, process, network, and topology coverage out of the box
- Smartscape dependency map and Davis AI tuned to its data model

- RUM-to-backend trace correlation
- Centrally managed updates — no application redeploy
- Operations-team-friendly (instrument an environment without engineering involvement)

OneAgent — cons

- Locked to Dynatrace as a backend
- Does not run on serverless functions
- Does not inject Go or Ruby application code (still adds host/process value)
- Default thread-local-style context model fragments under some async runtimes (effect systems, certain coroutine layouts)
- Custom-span surface (OneAgent SDK) is intentionally narrow — no metrics or log signals; not available for Python, Go, PHP, Ruby
- Framework support is broad but not exhaustive (e.g., http4s, Tapir, niche message brokers, exotic RPC layers)

OpenTelemetry — pros

- Vendor-neutral; works against Dynatrace, Jaeger, Tempo, New Relic, Datadog, Splunk, Honeycomb, Grafana Cloud, etc.
- Full signal coverage — traces, metrics, logs, baggage
- Per-runtime context-aware libraries (otel4s, zio-telemetry, kotlinx-coroutines, contextvars, context.Context, AsyncLocal, AsyncLocalStorage) handle each runtime's async model correctly
- First-class on serverless across every major runtime
- First-class on Go and Ruby where OneAgent doesn't inject application code
- Strong community ecosystem, monthly releases, broad plugin coverage
- Code-level control of sampling, attributes, and span structure

OpenTelemetry — cons

- No automatic host / runtime / topology coverage without a Collector or supplementary agent
- No equivalent to Smartscape or PurePath without backend enrichment
- Requires application redeploy for SDK changes
- Quality of Davis AI and other Dynatrace platform features depends on attribute completeness
- More moving parts (auto-instrumentation, SDK code, Collector) than a single OneAgent install
- Coverage breadth varies per runtime — JVM and .NET have the deepest OTel coverage; Ruby and PHP are still maturing

> **Sources:**
>
> - Technology support matrix (DT docs) — OneAgent framework, host, runtime coverage
> - Use OneAgent with OpenTelemetry (DT docs) — Smartscape/PurePath/Davis as OneAgent-only platform features
> - OpenTelemetry main site (opentelemetry.io) — vendor neutrality, signal coverage, ecosystem
> - OpenTelemetry Java SDK (OpenTelemetry GitHub) — monthly minor releases

12. Is the Conversion Worth the Effort?

Question every customer should ask before starting: *"What capability am I gaining that I do not already have, and is it worth the engineering disruption to get it?"*

Run through this checklist honestly:

1. **Does my OTel data already land in Dynatrace via OTLP?** If yes, you already have most of what "converting" would give you for traces.
2. **Am I missing host, runtime, or topology signals?** If yes, *adding* OneAgent solves that — *removing* OTel does not.
3. **Am I on an async-fragmenting runtime?** (Scala effect systems, certain coroutine layouts.) If yes, removing the runtime-aware OTel layer will *break* tracing — there is no OneAgent equivalent.
4. **Am I on Go or Ruby?** If yes, there is no OneAgent SDK to replace OTel with. Conversion is not technically possible at the application-span level.
5. **Do I have any serverless workloads?** If yes, OneAgent cannot run there; you will need OTel anyway, and a hybrid is unavoidable.
6. **Is there an organizational mandate for vendor portability or multi-backend support?** If yes, OTel is a requirement, not a choice.

7. **What is the cost of a coordinated multi-service rewrite of custom spans, metrics, and logs?** Compare honestly against the cost of running both for the foreseeable future.
8. **Is anyone on the team going to maintain a parallel internal instrumentation library against the OneAgent SDK?** If not, even partial Path C is a slow walk into instrumentation rot.

Common false economies:

- *"Removing OTel will reduce overhead."* In community practice, OTel auto-instrumentation overhead is small enough that removing it after installing OneAgent does not measurably move the needle. OneAgent has its own overhead in the same general range. Exact percentages depend heavily on workload, sampling rate, and exporter configuration — measure in your environment rather than relying on a published number.
- *"One instrumentation model is simpler."* True at the code level, false at the platform level — Dynatrace was designed to merge both, and customers running both report fewer surprises than customers running one tool stretched beyond its design center.
- *"OneAgent traces are better than OTel traces."* For overlapping framework coverage on standard stacks, the data is comparable. OneAgent's advantage is in the *surrounding* signals (host, runtime, topology, Davis), not in the raw spans.

The honest answer for almost every team already on OTel: the level of effort to *add* OneAgent is small (a deployment-engineering task, not a re-instrumentation task). The level of effort to *replace* OTel with OneAgent is large — and impossible in some runtimes — and almost never repaid by the marginal capability gained. **Layer, don't convert.**

> **Sources:** OneAgent SDK for Java (Dynatrace GitHub) — what the SDK does and does not cover, OneAgent SDK for .NET (Dynatrace GitHub) — metrics/logs out of scope, Use OneAgent with OpenTelemetry (DT docs).
>
> The §12 overhead range and dual-tooling observation are framed as **community guidance** — vendor-published numbers were not located. Measure overhead in your own environment.

13. Recommended Approach

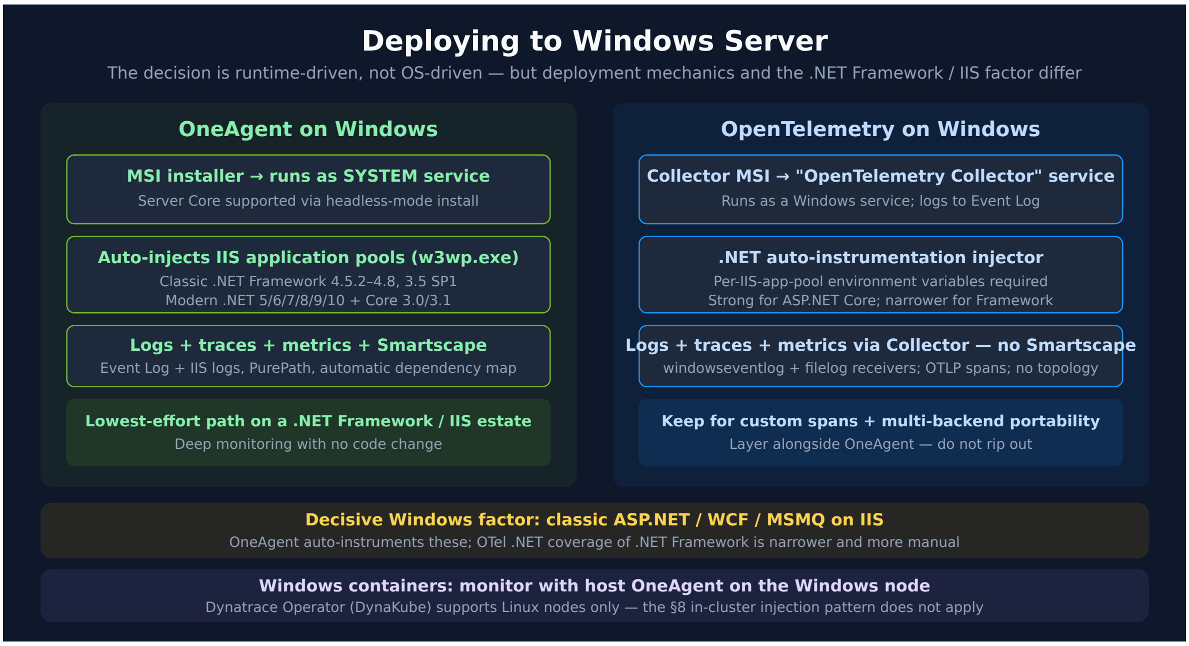
Default recommendation for any backend already instrumented with OpenTelemetry:

1. **Keep your OpenTelemetry instrumentation in place.** Do not start a conversion project.
2. **Install OneAgent** on the hosts (or via the Dynatrace Operator on Kubernetes) where the services run. Restart the processes. (For Go or Ruby workloads, OneAgent still adds host/process/Smartscape value even though it doesn't inject application code.)
3. **Pick one of the two supported coexistence patterns** to avoid duplicate spans:
   - *Pattern 1 — Span Sensor:* enable the OpenTelemetry Span Sensor in the OneAgent code module for your runtime; **disable** any OTLP exporter in the application. OneAgent picks up the OTel API calls in-process. (Available where the OneAgent code module supports it — check per-runtime docs.)
   - *Pattern 2 — OTLP:* keep the OTel exporter pointed at Dynatrace's OTLP endpoint (or an OTel Collector forwarding to Dynatrace); **disable** the Span Sensor. Spans correlate via W3C `traceparent`. This is the only option for Go, Ruby, and serverless.
4. **For async-fragmenting runtimes** (Scala effect systems, certain coroutine layouts), ensure the OTel layer is the runtime-aware library (otel4s, zio-telemetry, kotlinx-coroutines instrumentation) rather than the raw default SDK. Do not attempt to replace this layer with the OneAgent SDK — even where the SDK exists, it has no equivalent of the runtime's async-context primitive.
5. **For serverless workloads**, OTel only. OneAgent does not run there.
6. **Do not rewrite custom OTel spans, metrics, or logs against the OneAgent SDK** unless you have a specific Dynatrace capability that the SDK uniquely enables (in practice, this is rarely the case — and the SDK doesn't exist for Python, Go, PHP, or Ruby).
7. **Re-evaluate annually** — if Dynatrace adds new async-aware tracing or richer SDK coverage in your runtime, the calculus may shift. Today, layering is the right answer.

> **Sources:**
> - Use OneAgent with OpenTelemetry (DT docs) — Span Sensor and OTLP coexistence patterns; duplicate-spans warning
> - OpenTelemetry on AWS Lambda (DT docs)
> - OneAgent SDK for Java (Dynatrace GitHub) — serverless-not-supported guidance
> - otel4s (Typelevel GitHub), zio-telemetry (ZIO GitHub)

14. Deploying to Windows Servers

Short answer: the framework applies to Windows Server unchanged. The OneAgent-vs-OTel decision is driven by *runtime* (Java / .NET / Node / Python / Go / PHP / Ruby) and *deployment model* (long-lived host vs serverless vs async-fragmenting), **not by the operating system**. A Java service gets the same answer on Windows as on Linux. What changes on Windows is the *weighting* of a few factors and the *deployment mechanics* — covered below.



OneAgent is a first-class citizen on Windows Server

OneAgent installs from an **MSI package and runs as a Windows `SYSTEM` service** — the install creates the registry entries that start it at boot. Server Core installations are supported in headless mode. It delivers the same full-stack deep monitoring it does on Linux: host metrics, process detection, runtime metrics, log streams, Smartscape topology, and PurePath — none of which require a code change. For the current supported Windows Server version list (and Server Core / LTSC notes), check the OneAgent OS support matrix linked in the Sources block rather than hard-coding a version here — the matrix moves with each release train.

The decisive Windows factor — .NET Framework and IIS

This is the one place where "which OS" materially changes the recommendation, because the classic, Windows-only **.NET Framework** stack lives here.

- **OneAgent auto-instruments classic .NET Framework** (4.5.2–4.8 and 3.5 SP1 supported; 4.5–4.5.1 limited) **and modern .NET** (5/6/7/8/9/10 and Core 3.0/3.1), including **ASP.NET Framework, ASP.NET Core, WCF, gRPC, and named-pipe / TCP / MSMQ services**. IIS application pools have built-in instrumentation rules — OneAgent injects into the `w3wp.exe` worker processes automatically.
- **OTel's .NET auto-instrumentation is strongest for ASP.NET Core** and is narrower, more manual, and version-sensitive for classic ASP.NET Framework. On IIS it requires per-application-pool environment-variable wiring.

The practical consequence: **on an estate that is predominantly legacy .NET Framework + IIS, OneAgent is the decisive low-effort path to deep coverage** — it instruments the classic stack that OTel covers least well, with no code change and no per-app-pool wiring. OpenTelemetry remains the right layer for *custom business spans* and for *multi-backend portability*; layer it alongside OneAgent (per §8), do not rip it out. For modern ASP.NET Core workloads the coverage gap closes and the choice reverts to the general framework in §9.

Three 1.343 additions that close .NET gaps this section leans on

The argument above — that OneAgent is the low-effort path on a .NET Framework / IIS estate — depends on how completely OneAgent covers the .NET publishing and hosting shapes actually in use. **OneAgent 1.343 (released 07/28/2026)** closes three of them. As always, **verify the agent version on the hosts concerned**; a fleet on 1.342 or earlier does not have these yet.

- **.NET single-file self-contained apps are now monitored.** This is the load-bearing one. Single-file self-contained publish (`PublishSingleFile` with a bundled runtime) was a known instrumentation gap, which mattered precisely because it is a common way to ship a self-hosted .NET service onto a Windows box without installing a runtime. A team that had hit that gap would reasonably have concluded OneAgent did not cover their deployment style and reached for OTel — the decision logic in this section would have sent them the wrong way. From 1.343 it is covered.
- **gRPC status codes for .NET** are now captured, so a failing gRPC call carries the status rather than surfacing as an opaque error. Relevant to the WCF-replacement pattern, where gRPC is the usual modern successor on Windows service stacks.
- **Kong Gateway Enterprise 3.10–3.14** is supported, which matters where a Windows-hosted .NET estate sits behind Kong rather than IIS ARR or a hardware load balancer.

None of this changes the section's conclusion — it strengthens it, by removing the exceptions that used to argue against it.

Logs and traces on Windows

Both tools deliver **logs and traces** on Windows — the difference is configuration effort and correlation fidelity.

Logs

- **OneAgent** auto-detects and ingests the **Windows Event Log** channels (System / Application / Security) and **text log files** (IIS logs, custom application logs). It is enabled by the *[Built-in] Windows system, application, and security logs* log-ingest rule, or included automatically under *[Built-in] Ingest all logs*. Each log line is auto-correlated to the host / process / service entity that produced it.
- **OpenTelemetry-only** reads the Event Log via the Collector's `windowseventlogreceiver` (the `channel:` field is required — e.g. `Application`, `System`, `Security`) and text logs via the `filelog` receiver, then exports OTLP. You configure each channel and path and own the attribute hygiene that ties logs back to entities.

Traces

- **OneAgent** PurePath auto-captures traces for .NET (including IIS `w3wp.exe`), Java, and Node on Windows with no code change. Critically, it traces the classic Windows service stack — **ASP.NET Framework, WCF, gRPC, and named-pipe / TCP / MSMQ services** — which OTel's .NET auto-instrumentation covers least well.
- **OpenTelemetry-only** spans come from the .NET auto-instrumentation injector / SDK and export via OTLP; they correlate with OneAgent spans through W3C `traceparent` (per §8). Coverage of classic .NET Framework / WCF / MSMQ is narrower, so some Windows service hops can be missing unless manually instrumented.

Smartscape dependency mapping on Windows

This is the **sharpest OneAgent-vs-OTel difference on Windows**, and the reason to run OneAgent for topology even when OpenTelemetry owns the spans.

Dependency-mapping aspect	OneAgent on Windows	OpenTelemetry-only on Windows
Topology graph	Automatic Smartscape — host → process group → service, real-time	None — no Smartscape topology graph
Service-to-service dependencies	Detected automatically for every instrumented service	Inferable only from span parent/child of *instrumented* calls
Process / host entity model	Automatic — every Windows process and host is an entity	Not produced; must be approximated from span attributes
Windows service-stack hops (ASP.NET Framework, WCF, MSMQ, named-pipe / TCP services)	Captured because OneAgent auto-instruments these (§6) → they appear in the map	Appear only if explicitly instrumented; classic Framework / WCF / MSMQ coverage is weak
Uninstrumented / third-party processes	Still placed on the map via host / process observation	Invisible — no span, no node
Configuration effort	Zero	Per-service instrumentation + attribute hygiene

Practical impact for a Windows estate: IIS + WCF + MSMQ + named-pipe IPC is exactly the interaction style OpenTelemetry instruments least completely. An OTel-only deployment shows the hops you instrumented and a blank where you didn't — there is no automatic host / process topology to fall back on. OneAgent fills that gap with a complete, zero-config dependency map. If Smartscape dependency mapping matters to you (impact analysis, Davis RCA, blast-radius), **run OneAgent for topology and let OpenTelemetry own custom spans** — the §13 "layer, don't convert" recommendation applied to the dependency-map question.

Deployment mechanics on Windows

Concern	OneAgent	OpenTelemetry
Install unit	MSI; runs as a `SYSTEM` Windows service (`--unpack-msi` extracts the package + batch installer for scripted rollout)	Collector MSI registers a Windows service named "OpenTelemetry Collector" and an Event Log source; per-app .NET injector for in-process instrumentation
IIS-hosted apps	Auto-detected; instruments the application pools' `w3wp.exe` processes — no per-pool config	.NET auto-instrumentation requires environment variables set on each IIS application pool
App redeploy	Not required — agent attaches at the next process restart	SDK changes need a redeploy; auto-instrumentation changes need a process restart
Host / runtime metrics	Automatic	Add a `hostmetrics` receiver to a Collector running on the node
Coexistence	Use the §8 Span Sensor or OTLP pattern to avoid duplicate spans	Same — pick one ingestion path per process

Windows containers

Container monitoring on Windows is **host-based, not in-cluster-injected**. The Dynatrace Operator (DynaKube) supports **Linux worker nodes only** — there is no Windows-node DynaKube path. Monitor Windows-container workloads by installing **OneAgent on the Windows host node**, which observes the containers running on it; the in-container init-container injection model described in §8 does not apply to Windows containers. If your platform is mixed (Linux + Windows nodes), use the Operator for the Linux nodes and host OneAgent for the Windows nodes.

Each conclusion below is a synthesis, not a documented recommendation: the .NET Framework / IIS call rests on the documented OneAgent-vs-OTel coverage asymmetry, the Windows-container call on the Operator being Linux-only plus the host-based Windows-container model, and the Smartscape rows on OneAgent's documented .NET WCF / MSMQ / named-pipe instrumentation (§ 6) together with § 4's position that topology is a OneAgent-only construct — OTel emits spans, not topology.

> **Sources:**
> - Install OneAgent on Windows (DT docs) — *"Creates entries in the Windows Registry that start OneAgent as a `SYSTEM` service"*; `--unpack-msi` extraction for scripted install
> - .NET technology support (DT docs) — Framework 4.5.2–4.8 + 3.5 SP1 and modern .NET 5–10 + Core 3.0/3.1 supported; IIS application-pools have built-in instrumentation rules
> - OneAgent platform and capability support matrix (DT docs) — current supported Windows Server versions + Server Core / headless-mode notes
> - OneAgent 1.343 release notes (DT docs) — released 07/28/2026; .NET single-file self-contained app monitoring, gRPC status codes for .NET, Kong Gateway Enterprise 3.10–3.14
> - Install the Collector on Windows (opentelemetry.io) — MSI installs the Collector as a Windows service ("OpenTelemetry Collector") with an Event Log source
> - Get started with Kubernetes monitoring — Full-Stack (DT docs) — Dynatrace Operator targets Linux worker nodes
> - Windows event logs (DT docs) — OneAgent auto-detects System / Application / Security channels; enabled via the *[Built-in] Windows system, application, and security logs* rule or *[Built-in] Ingest all logs*
> - Windows Event Log Receiver (OpenTelemetry collector-contrib GitHub) — *"tails and parses logs from windows event log API"*; required `channel` field
> - Distributed traces — concepts (DT docs) — OneAgent auto-collects topology data and entity relationships for Smartscape; PurePath
> - **Derived:** every conclusion in this section combines a documented OneAgent/OTel coverage asymmetry with this entry's low-code-change thesis (§§ 4-6, 9); no page states them as recommendations

15. Runtime Applicability Map

Runtime-by-runtime — applicability of this guidance:

Runtime	Applies as written?	Notes
Java (classical threads, `CompletableFuture`, executors)	Yes	Reference case — OneAgent auto-instrumentation is strong
Java 21+ (Project Loom virtual threads)	Yes	Both OneAgent and OTel Java agent support virtual threads in current versions
Kotlin	Mostly	Coroutines have a fiber-like context flow; OTel's `kotlinx.coroutines` instrumentation handles it. OneAgent's coroutine support has improved but verify in your version
Scala (classical — Akka, Play, plain `Future` / `ForkJoinPool`)	Yes	OneAgent auto-instrumentation is strong here
Scala (effect systems — ZIO, Cats Effect, fs2, http4s on IO)	Modified	See §7 — OTel via *otel4s* / *zio-telemetry* is technically required, not preferred
Groovy	Yes	Tracks Java; auto-instrumented by both
Clojure	Mostly	Same JVM auto-instrumentation; `core.async` channels have context issues comparable to coroutines
.NET Framework (4.5+ / classical ASP.NET)	Yes	OneAgent auto-instruments; OTel coverage is narrower for ASP.NET Framework. On IIS / Windows Server this is the decisive factor — see §14
.NET 6 / 7 / 8+ (modern .NET)	Yes	Both have strong coverage; `AsyncLocal<T>` flows trace context natively across `async` / `await`
Node.js (JavaScript)	Yes	Both use `async_hooks` / `AsyncLocalStorage`
Node.js (TypeScript)	Yes	Same as JavaScript at runtime
Python (sync — Django, Flask)	Yes	Both have mature coverage
Python (asyncio — FastAPI, Starlette, Tornado)	Yes	Both use `contextvars`
Python (greenlet / gevent / eventlet monkey-patched)	Modified	Mixed coverage in both; verify per-library; OTel often the more reliable path
Go	OTel-only at app level	No Go OneAgent SDK; OTel uses explicit `context.Context`. OneAgent provides host/process/Smartscape only
PHP	Yes	OneAgent's PHP module is mature; OTel is newer but sufficient
Ruby	OTel-only at app level	Dynatrace's documented recommendation; OneAgent supplies host/process
Mobile native (iOS / Android)	Out of scope	Use Dynatrace Mobile Agent or OTel mobile SDKs — covered in the MOBL series
Frontend Web (browser JS)	Out of scope	Use Dynatrace RUM JavaScript or OTel browser SDK — covered in the WEBRUM series

Adjacent OTel ecosystem libraries to know about:

- **Java / Scala** — `opentelemetry-java-instrumentation` (Java agent), `otel4s` (Cats Effect-native), `zio-telemetry` (`zio-opentelemetry` module)
- **.NET** — `OpenTelemetry.Instrumentation.AspNetCore`, `OpenTelemetry.Instrumentation.Http`, `OpenTelemetry.Instrumentation.SqlClient`, `OpenTelemetry.Instrumentation.GrpcNetClient`
- **Node.js** — `@opentelemetry/auto-instrumentations-node`, `@opentelemetry/sdk-trace-node`, `@opentelemetry/context-async-hooks`
- **Python** — `opentelemetry-distro`, `opentelemetry-instrumentation-django`, `opentelemetry-instrumentation-fastapi`, `opentelemetry-instrumentation-aiohttp-client`
- **Go** — `go.opentelemetry.io/otel`, `otelhttp`, `otelgin`, `otelgrpc`, `otelpgx`, `otelmongo`
- **PHP** — `open-telemetry/opentelemetry-auto-laravel`, `open-telemetry/opentelemetry-auto-symfony`
- **Ruby** — `opentelemetry-instrumentation-all`, `opentelemetry-instrumentation-rails`, `opentelemetry-instrumentation-sinatra`

- **Cross-runtime** — `opentelemetry-collector` (vendor-neutral telemetry pipeline; common deployment pattern is app → Collector → Dynatrace OTLP)

Adjacent Dynatrace capabilities that depend on OneAgent:

- **Smartscape** — automatic topology and dependency mapping
- **PurePath** — code-level distributed trace with method-level granularity on demand
- **Davis AI** — anomaly detection, problem correlation, RCA
- **Real-User Monitoring ↔ backend** — RUM session-to-trace stitching
- **Process / host / log correlation** — automatic by virtue of agent placement

Adjacent topic series in this notebook collection:

- **OTEL series** — OpenTelemetry integration deep dives
- **AIOPS series** — Davis AI, predictive / causal / generative intelligence
- **K8S series** — Dynatrace Operator, DynaKube, Kubernetes observability
- **MOBL series** — native mobile monitoring (iOS / Android)
- **WEBRUM series** — browser RUM and Core Web Vitals
- **DASH series** — dashboards over mixed OneAgent + OTel data
- **ADOPT series** — instrumentation maturity and adoption roadmap

> **Sources:**
> - Technology support (DT docs)
> - Java technology support (DT docs) — JVM detail (Akka/Pekko, Play, Loom)
> - Go OpenTelemetry walkthrough (DT docs) — the documented Go path is OpenTelemetry instrumentation; the page describes no OneAgent code-level injection for Go
> - OneAgent SDK for Java (Dynatrace GitHub) — serverless-not-supported
> - OpenTelemetry on AWS Lambda (DT docs) — Lambda extension covers Python/Node/Java
> - otel4s (Typelevel GitHub), zio-telemetry (ZIO GitHub) — basis for *"Modified"* applicability rating on Scala effect systems
>
> The Clojure `core.async` note is **community guidance**, not vendor-documented.
>
> **A signal worth tracking for Rust (Dynatrace API 1.346, released 08/25/2026).** The API changelog adds **`RUST` and `TOKIO`** as `serviceTechnology` values on the Request Attributes endpoints and as technology values on `GET /extensions/{technology}/availableHosts` (Early Access).
>
> Read this carefully, because it is a **configuration-surface** change, not an announcement of OneAgent auto-instrumentation for Rust. What it establishes is that Dynatrace now models Rust and Tokio as first-class technology values — which is a prerequisite for deeper support, and useful today if you are naming or attributing Rust services. For the decision this entry frames, **Rust remains an OpenTelemetry-instrumented runtime**: it is absent from the auto-instrumentation coverage in §6 and the async-context-propagation matrix in §7, and nothing in 1.346 changes that. Treat this as a reason to re-check the supported-technologies page over the next few sprints, not as a reason to revisit an OTel decision for Rust today.
>
> **Sources:** Dynatrace API changelog version 1.346 (DT docs) — `RUST` and `TOKIO` added to the serviceTechnology and extension-technology enums, read 08/28/2026.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against the current Dynatrace documentation.